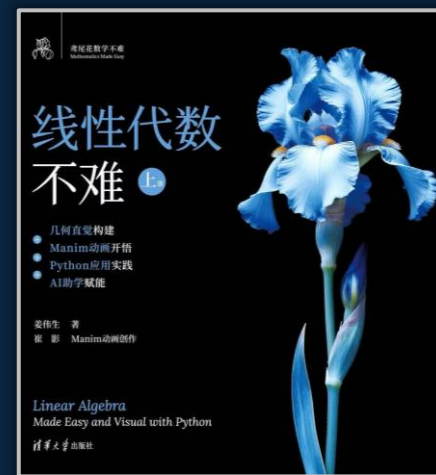


02 矩阵

2.2 转置

A^T 记法、行列互换与转置的重要性质



鸢尾花书·《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1

理解矩阵转置的定义：将矩阵的行、列互换得到的新矩阵，记作 A^T

2

掌握向量转置：行向量转置成列向量，列向量转置成行向量

3

理解不同形状矩阵转置后的形状变化：细高 $\leftrightarrow$ 扁平、方阵不变、对称矩阵不变

4

熟练使用 NumPy 的 `.T` 获取二维数组的转置，并用热图可视化转置前后的变化

5

从几何视角理解转置：行、列向量在不同维度空间中的箭头如何相互对应

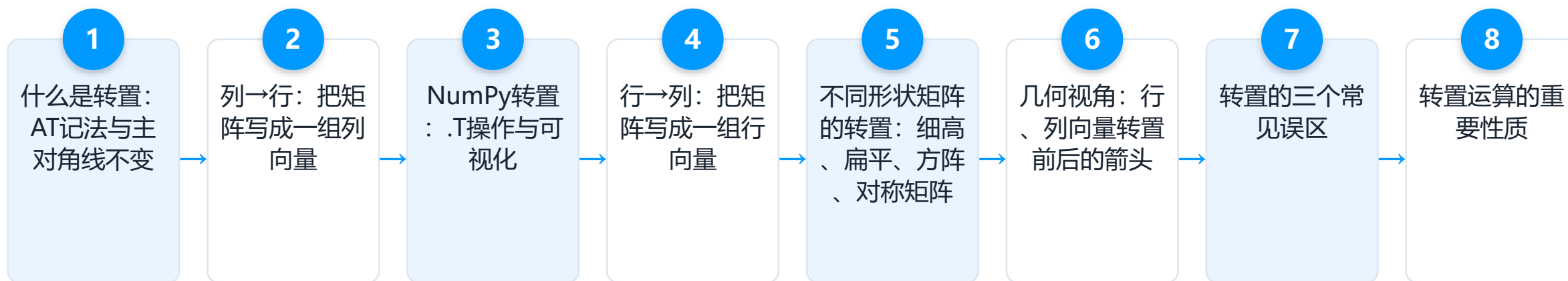
6

避开转置的三个常见误区：不是对角线镜像、不是左右镜像、不是上下镜像

7

掌握转置运算的重要性质：双重转置恢复原矩阵、对加法可交换、数乘不受影响

本节知识脉络



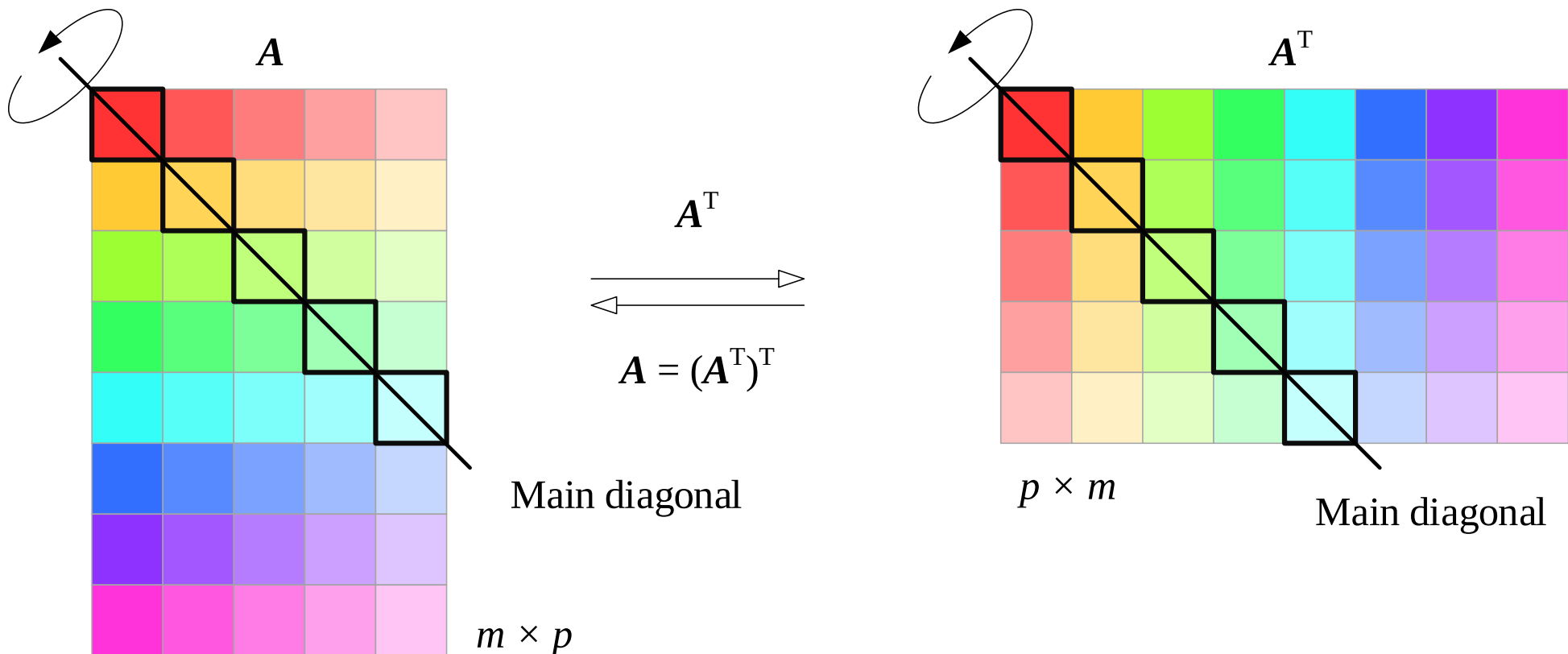
什么是转置？

矩阵 $\mathbf{A}$ 的转置记作 $\mathbf{A}^T$ ：把矩阵的行、列互换，得到一个新矩阵

- 列向量转置后得到行向量，反之亦然；转置是一种“矩阵→矩阵”的运算
- NumPy用 `.T` 表示数组转置运算，本书统一采用 $\mathbf{A}^T$ 记法（不用 $\mathbf{A}'$ ）
- $m \times p$ 矩阵 $\mathbf{A}$ 转置后得到 $p \times m$ 矩阵 $\mathbf{A}^T$ ；主对角线上的元素 $a_{i,i}$ 转置前后位置不变

图1 • 矩阵转置：主对角线不变

- 矩阵A绕主对角线镜像，得到转置矩阵 A^T ；对角线上的元素位置不变
- $m \times p$ 矩阵A转置后变为 $p \times m$ 矩阵 A^T



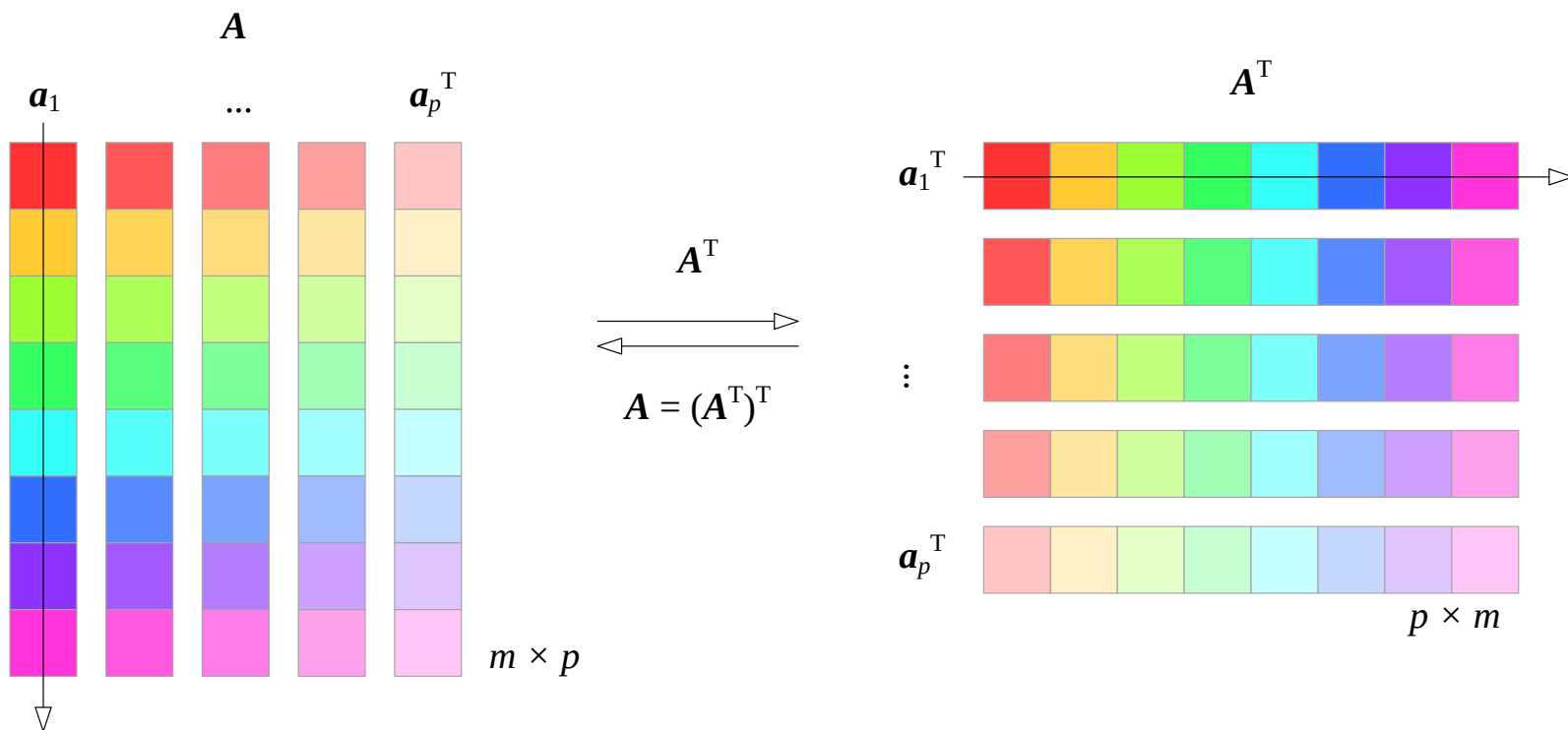
把矩阵写成一组列向量

$$A = [\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_p], \text{ 转置后 } A^T = [\mathbf{a}_1^T; \mathbf{a}_2^T; \cdots; \mathbf{a}_p^T]$$

- 把矩阵A表示为一组列向量后，转置就是把每个列向量变成对应的行向量
- 矩阵A的第1列转置后成为 A^T 的第1行，第2列变成第2行，依此类推；元素的相对顺序不变

图2 • 转置把列变成行

- 每个原来的列向量，转置后变成 A^T 中对应的行向量
- 元素的相对顺序（图中色调的深浅排列）保持不变



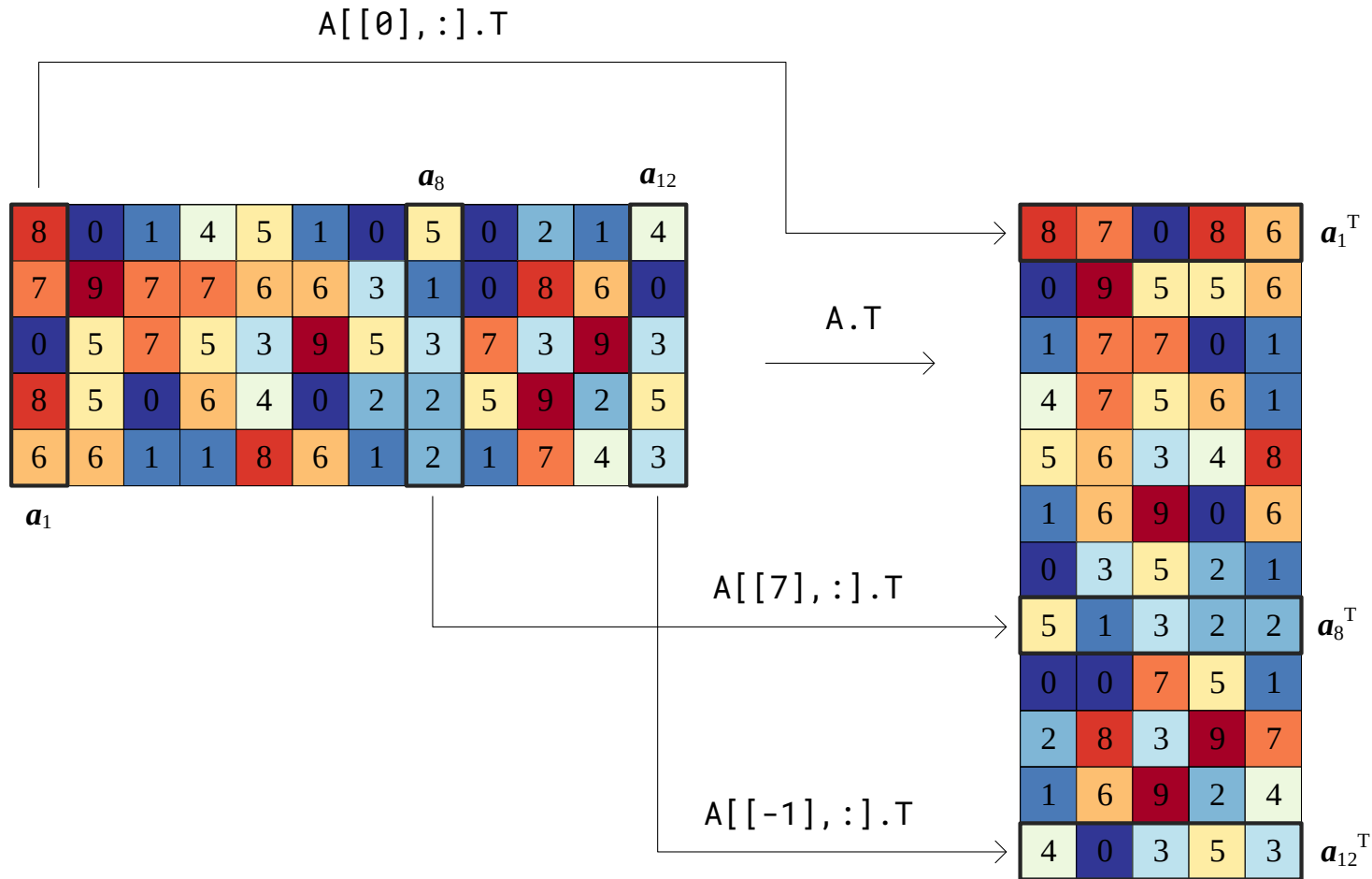
举例：把矩阵按列转置

$$A^T = [0,5; 3,4; 5,0]^T = [0,3,5; 5,4,0]$$

- 原矩阵A每列的第1个元素，转置后变成 A^T 每行的第1个元素
- 原矩阵A每列的第2个元素，转置后变成 A^T 每行的第2个元素

图3 • NumPy转置图解：列→行

- $A[[0],:].T$ 、 $A[[7],:].T$ 、 $A[[-1],:].T$ 分别把第1、8、12列取出并转置为对应的行
- $A.T$ 对整个矩阵做转置，得到形状为 12×5 的新数组



NumPy二维数组的转置：.T

用 `.T` 获取二维数组的转置，并用热图对比转置前后

- `np.random.seed(88)` 固定随机种子，`np.random.randint(0,10,(5,12))` 生成一个 5×12 的随机整数矩阵A
- `A_T = A.T` 一步得到矩阵A的转置；这是NumPy中获取转置最常用的方式
- `sns.heatmap(A[[0],:],...)` 可视化矩阵A的第1行；`sns.heatmap(A[[0],:].T,...)` 展示转置后同一行变成列向量
- `cmap`、`square`、`cbar`、`annot`、`vmin/vmax`、`linecolor`、`linewidths` 等参数控制热图的配色、格子形状与标注样式

初始化

```
import numpy as np
import seaborn as sns
```

a

```
np.random.seed(88)
A = np.random.randint(0, 10, (5, 12))
```

计算矩阵 A 的转置

b

```
A_T = A.T
```

行向量的转置

c

```
sns.heatmap(A[[0],:], cmap = 'RdYlBu_r', square = True,
            cbar = False, annot = True, vmin = 0, vmax = 9,
            linecolor = 'w', linewidths = 0.25)
```

d

```
sns.heatmap(A[[0],:].T, cmap = 'RdYlBu_r', square = True,
            cbar = False, annot = True, vmin = 0, vmax = 9,
            linecolor = 'w', linewidths = 0.25)
```



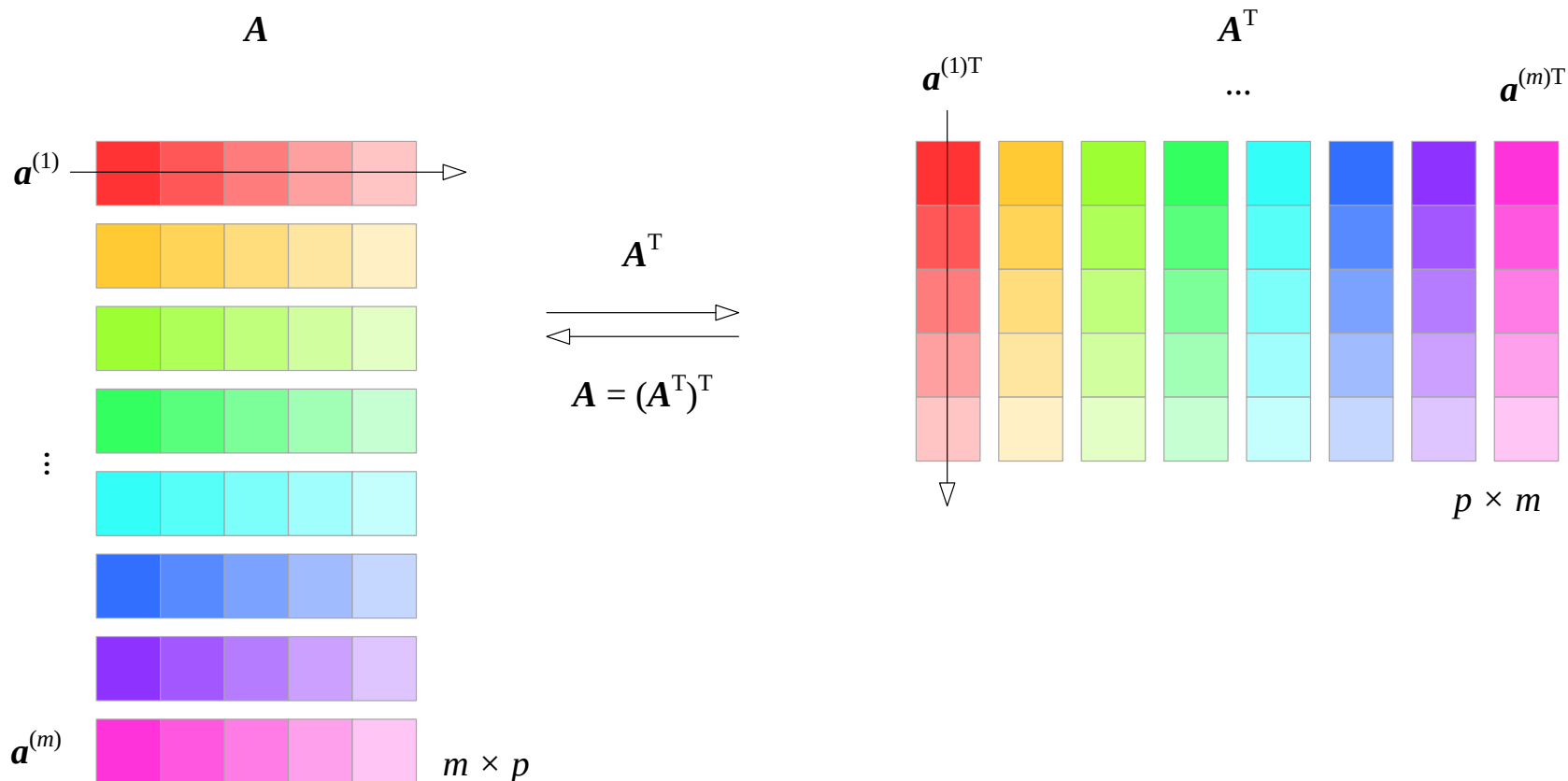
把矩阵写成一组行向量

$$\mathbf{A} = [\mathbf{a}^{(1)}; \mathbf{a}^{(2)}; \dots; \mathbf{a}^{(m)}], \text{ 转置后 } \mathbf{A}^T = [\mathbf{a}^{(1)T} \ \mathbf{a}^{(2)T} \ \dots \ \mathbf{a}^{(m)T}]$$

- 把矩阵A表示为一组行向量后，转置就是把每个行向量变成对应的列向量
- 矩阵A的第*i*行转置后成为A^T的第*i*列；元素的相对顺序（同一色调的深浅渐变）不变

图4 • 转置把行变成列

- 每个原来的行向量，转置后变成 A^T 中对应的列向量
- 矩阵 A 第 i 行的所有元素，都会变成 A^T 第 i 列的对应元素



举例：把矩阵按行转置

$$A^T = [0,5; 3,4; 5,0]^T = [0,3,5; 5,4,0]$$

- 原矩阵A每行的第1个元素，转置后变成 A^T 每列的第1个元素
- 原矩阵A每行的第2个元素，转置后变成 A^T 每列的第2个元素

图5 • NumPy转置图解：行→列

- $A[:, [0]].T$ 把矩阵的第1列取出并转置为行向量，展示转置的另一种视角
- LA_02_02_01.ipynb 还可可视化了矩阵A的行向量转置为列向量的过程

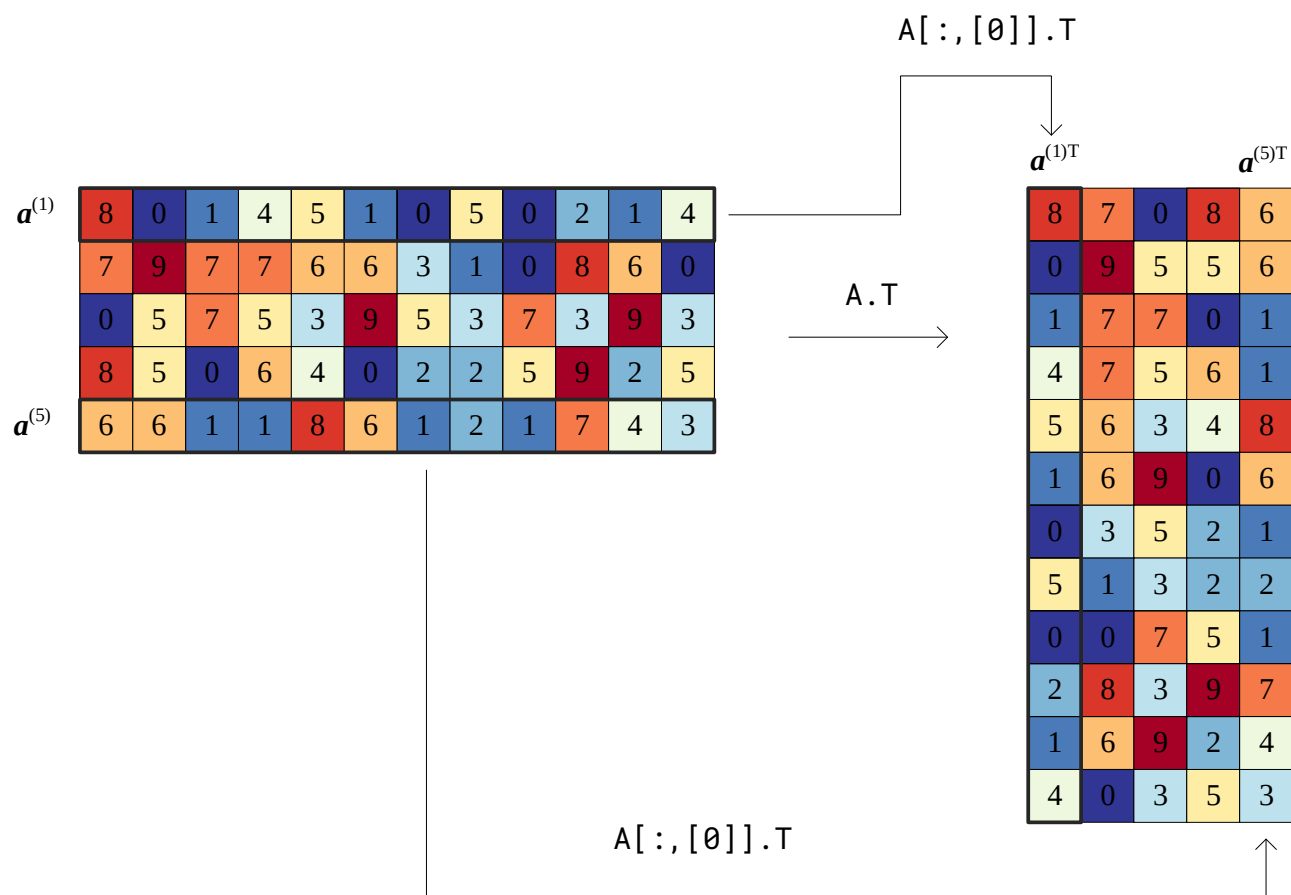
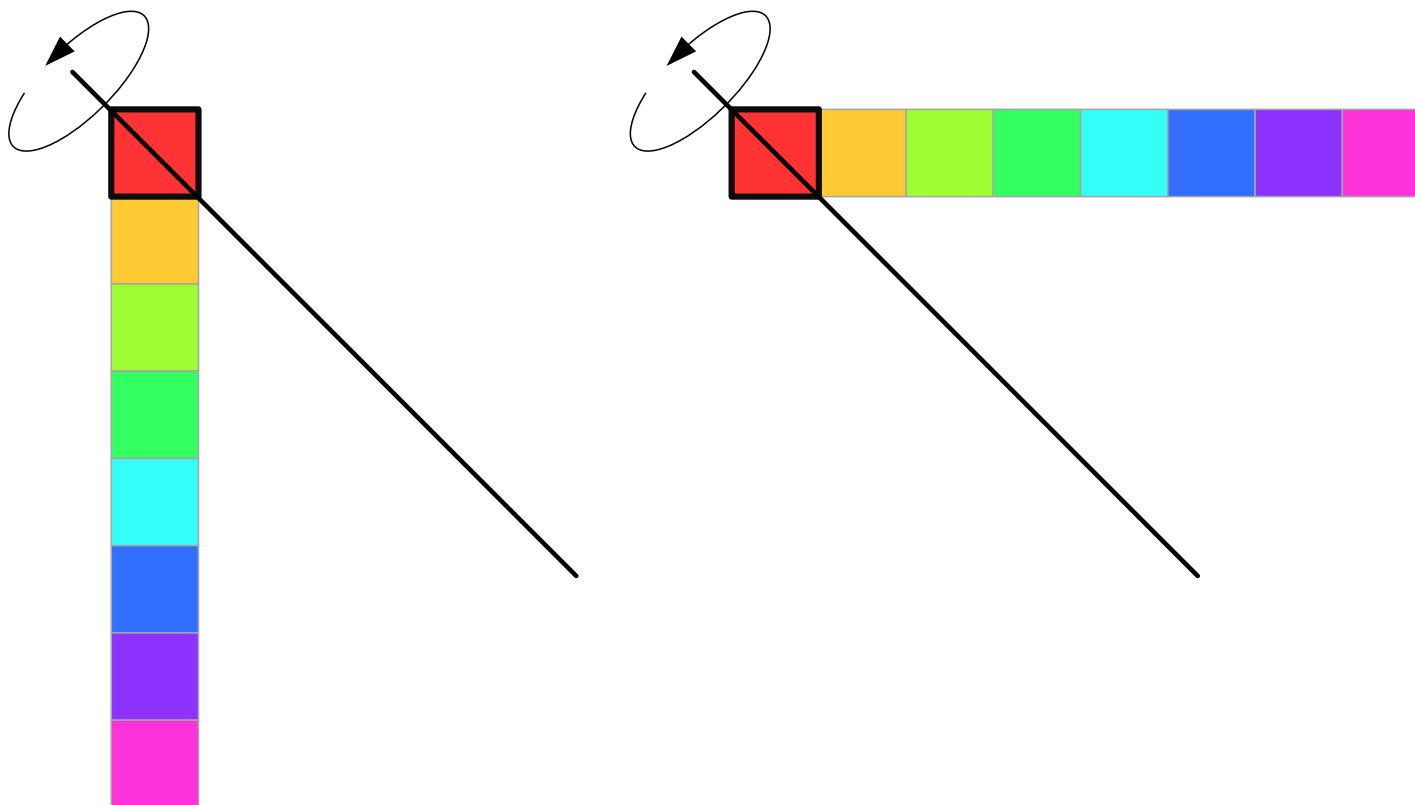


图6 • 行向量与列向量的相互转置

- 行向量转置得到列向量，例如 $[1,2,3]^T = [1;2;3]$
- 列向量转置得到行向量，例如 $[1;2;3]^T = [1,2,3]$



细高矩阵与扁平矩阵：转置互变

细高矩阵（行数多于列数）转置得到扁平矩阵（行数少于列数），反之亦然

- 如图1所示， $m \times p$ 矩阵转置后变为 $p \times m$ 矩阵；行、列数量互换，形状随之改变
- 核心规则不变：转置永远是“行变列，列变行”，元素间的相对位置不变

图7 • 方阵的转置仍是方阵

- 方阵（行数等于列数）转置后形状不变，仍是方阵
- 但元素位置会绕主对角线镜像，例如 $[1,3;2,4]^T = [1,2;3,4]$

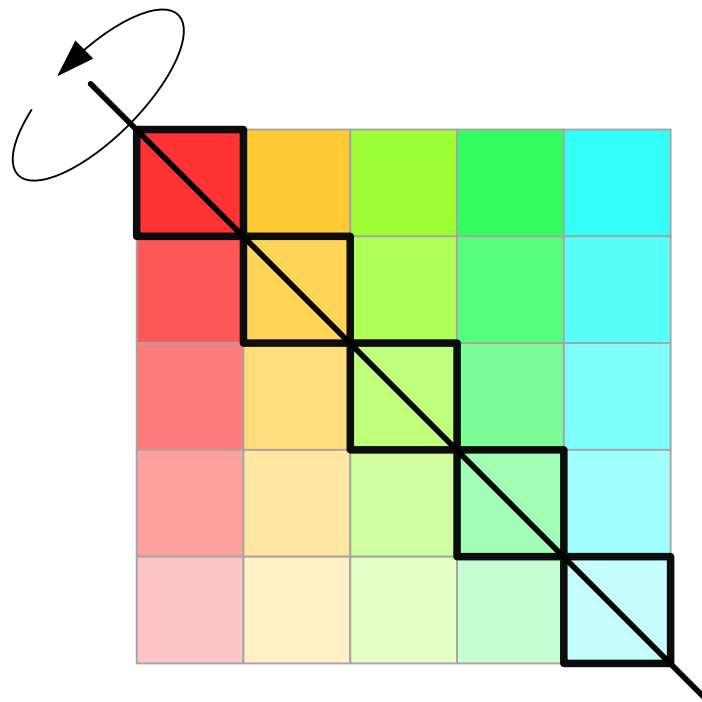
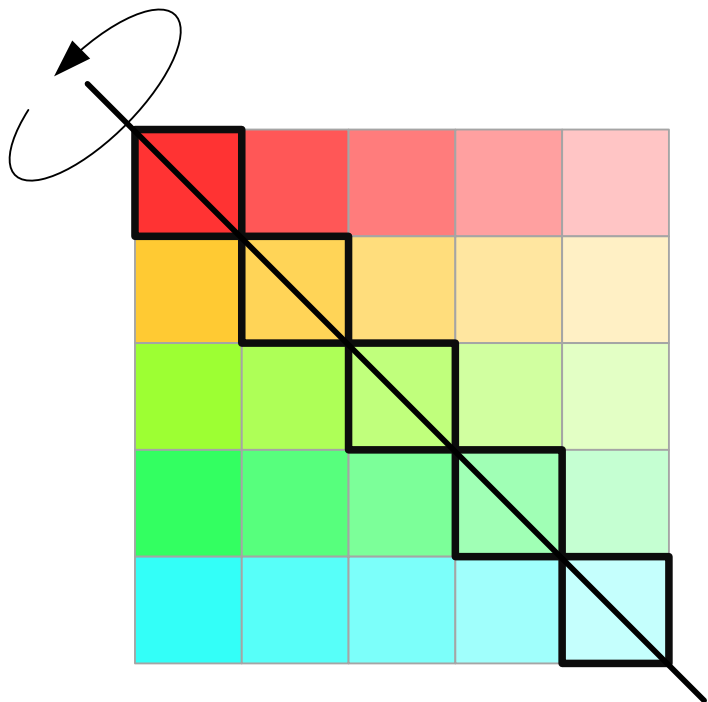


图8 • 对称矩阵转置不变

- 对称矩阵 (symmetric matrix) 满足 $A = A^T$, 转置后与自身完全相同
- 矩阵的形状是下一节 (2.3 矩阵的形状) 要探讨的话题

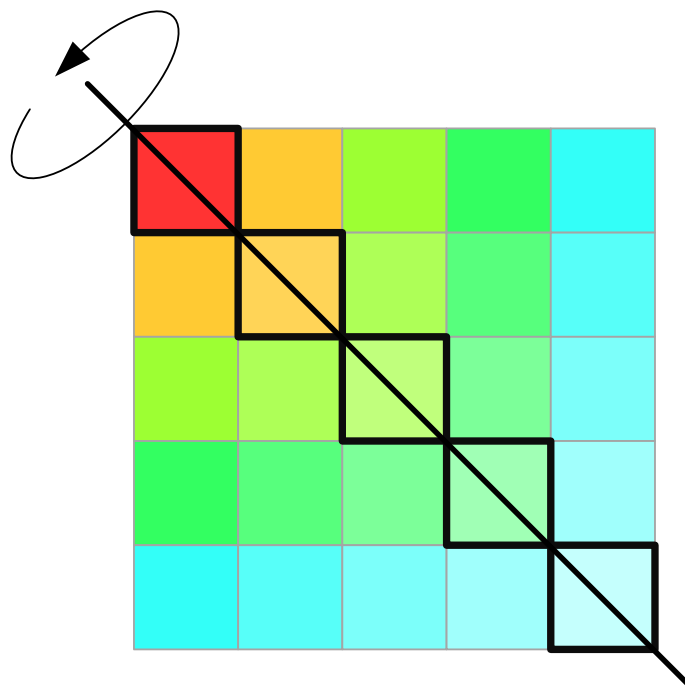
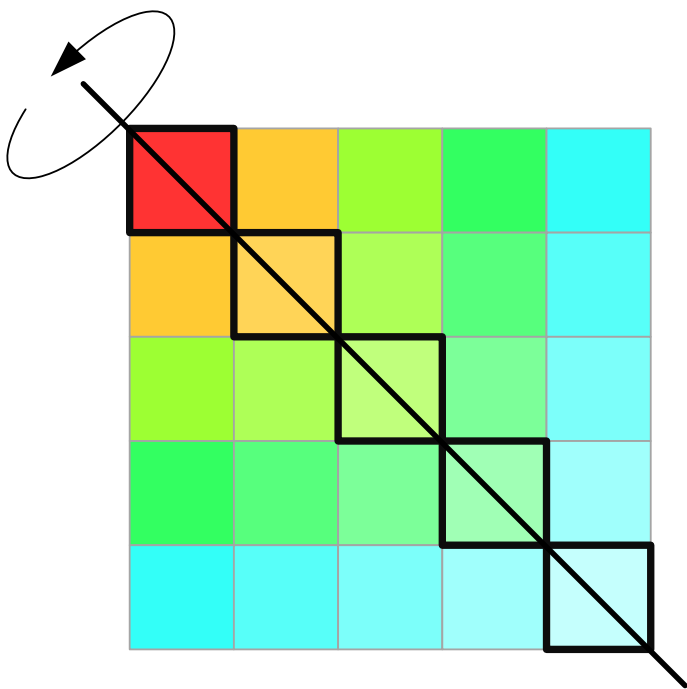


图9 • 矩阵A的行、列向量（几何视角）

- A的行向量可看成平面中的箭头，A的列向量可看成三维空间中的箭头
- 图片来自《可视之美》，展示矩阵与几何向量之间的对应关系

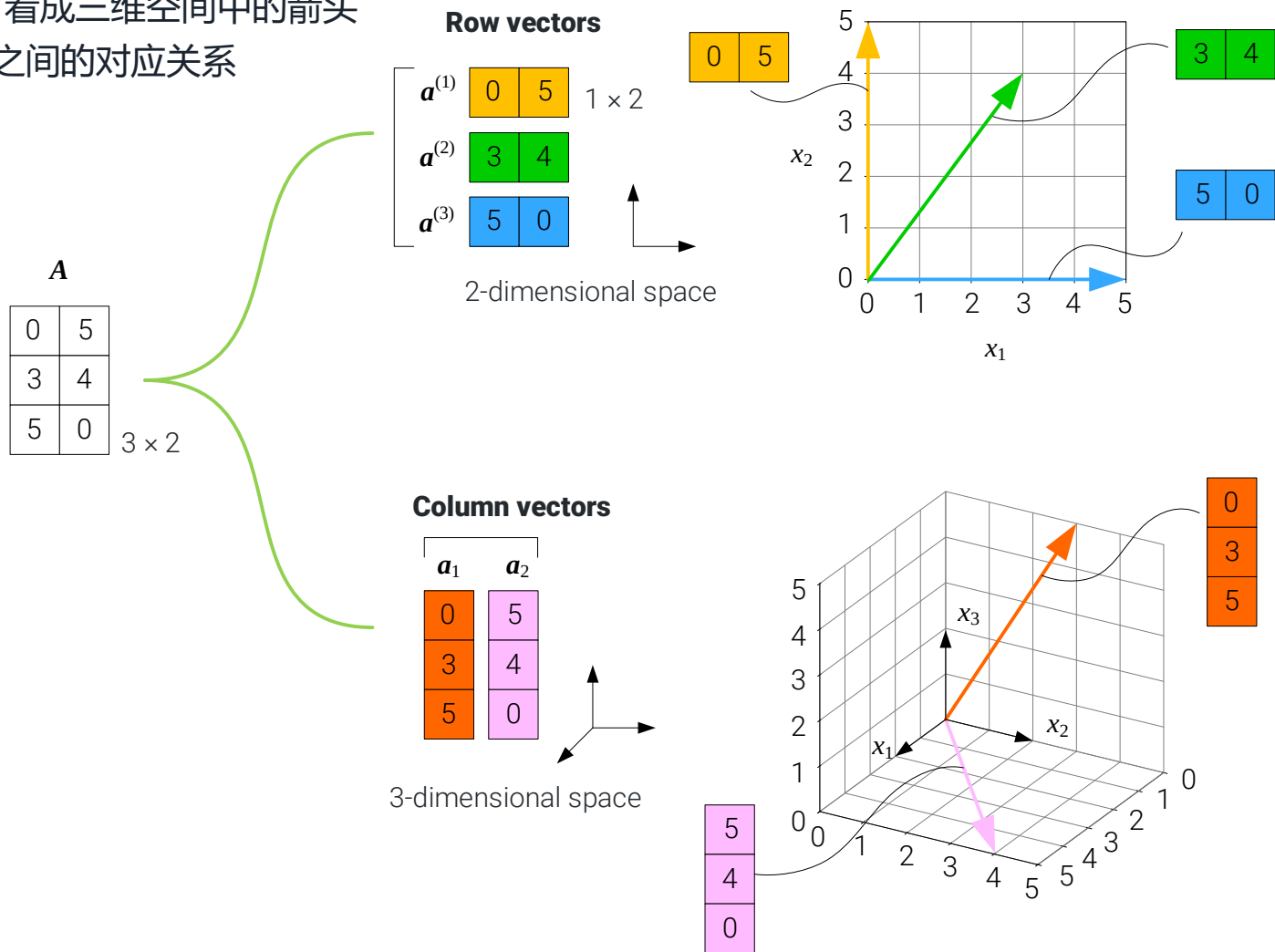


图10 • A^T 的行、列向量（几何视角）

- 转置之后，原来的列向量变成行向量： A^T 的行向量是三维空间中的箭头
- A^T 的列向量则变成平面中的箭头，与A的行向量一一对应

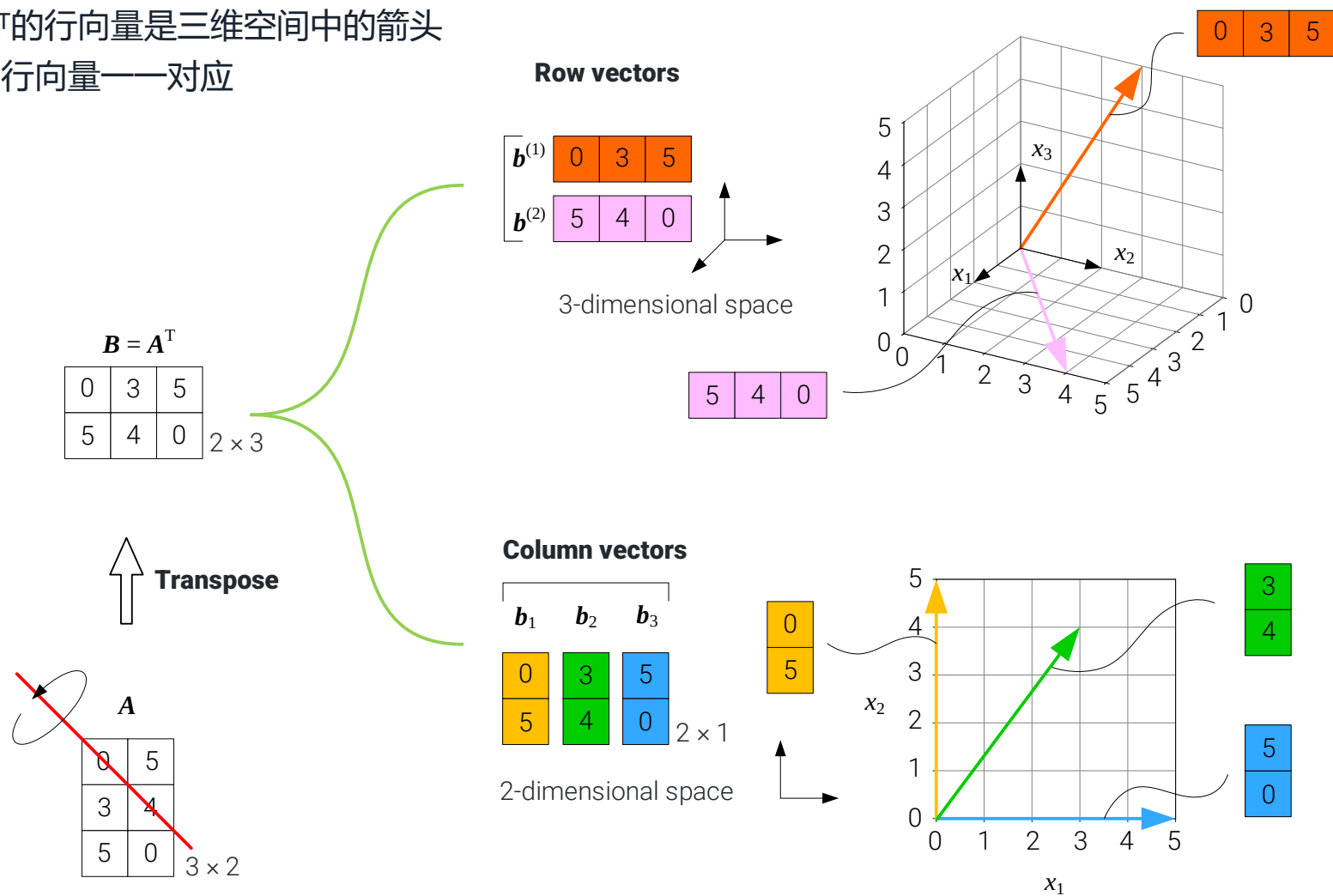
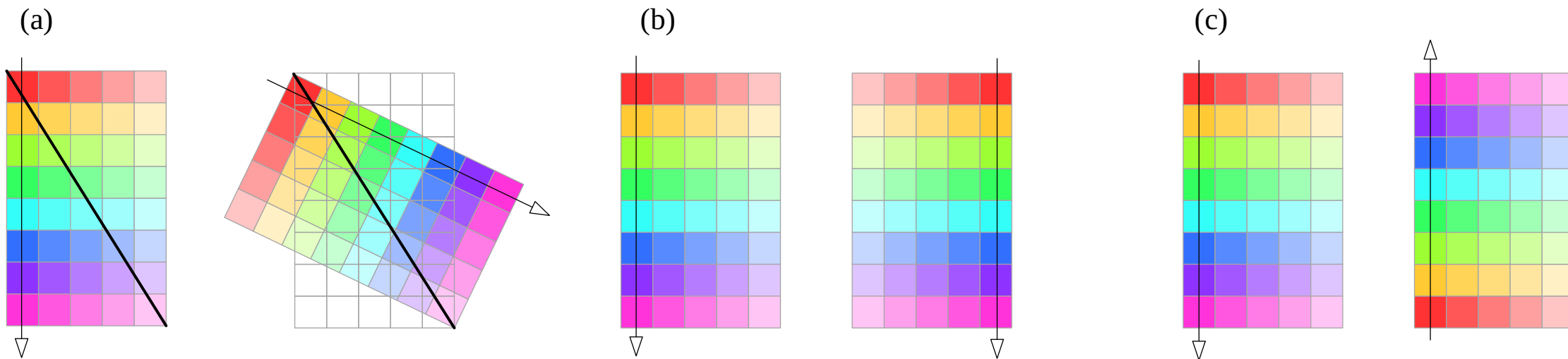


图11 • 转置的三个误区

- (a)(b)(c) 转置不是对角线镜像、不是左右镜像、也不是上下镜像
- 转置的本质只有一个：严格地将第 i 行的元素调整到第 i 列



性质一：双重转置恢复原矩阵

$$(A^T)^T = A$$

- 转置的定义是行变列、列变行；对已转置的矩阵再转置一次，就会恢复原来的行列结构
- 例如 A 转置得到 A^T ， A^T 再转置又得到 A

性质二：转置对加法可交换

$$(A + B)^T = A^T + B^T$$

- 矩阵加法按元素相加，转置只改变元素位置，因此先加后转置与先转置后加，结果相同
- 这一性质要求 A 、 B 形状相同；矩阵减法同样成立

性质三：数乘对转置无影响

$$(kA)^T = kA^T$$

- 标量 k 对矩阵的所有元素做同样缩放，转置只改变元素位置、不改变数值，二者可以交换顺序
- 转置与矩阵乘法、逆矩阵结合后还有更多性质，留待后续章节展开

本节关键术语

转置 *Transpose* — 将矩阵的行、列互换得到的新矩阵，记作 A^T

主对角线不变 转置前后位置不变的一组元素 $a_{i,i}$ 所在的连线

列向量的转置 列向量转置后得到行向量

方阵的转置 形状不变，元素位置绕主对角线镜像

双重转置 $(A^T)^T = A$

AT记法 本书统一用 A^T 表示转置，对应 NumPy 中的 $.T$

行向量的转置 行向量转置后得到列向量

细高矩阵、扁平矩阵 *Tall/Wide Matrix* — 转置后二者相互转换

对称矩阵 *Symmetric Matrix* — 满足 $A = A^T$ 的方阵

转置的分配律 $(A+B)^T = A^T+B^T$; $(kA)^T = kA^T$

配套代码与延伸阅读

代码1 • LA_02_02_01.ipynb

NumPy转置操作，列→行、行→列可视化

延伸阅读 • LA_02_02_02.ipynb

不同形状矩阵的转置，建议自学

推荐阅读 • 《可视之美》

本节图9、图10几何视角配图来源

开源仓库

全书代码与图像素材：<https://github.com/Visualize-ML>

习题 Q1 – Q9

Q1 LA_02_01_01.ipynb反复调用seaborn.heatmap()绘制类似热图，请将重复代码写成一个Python函数，方便后续可视化调用。

Q3 请编写Python代码验证“双重转置恢复原矩阵”；判断矩阵是否相同，请用numpy.array_equal()。

Q5 请编写Python代码验证“数乘对转置无影响”。

Q7 用NumPy生成一个 6×3 的随机整数矩阵A，编写代码验证A.T.shape 是否等于 (A.shape[1], A.shape[0])。

Q9 用随机数生成一个 5×5 矩阵A，构造 $B = A + A.T$ ，验证B是否一定是对称矩阵，并说明原因。

Q2 请用随机数生成器创建一个 5×8 矩阵，然后提取这个矩阵的每一行；最后，再用numpy.block()把这些行向量合成为原始矩阵。

Q4 请编写Python代码验证“转置运算对矩阵加法可交换”。

Q6 请编写Python尝试复刻图9、图10。

Q8 编写函数check_symmetric(A)，判断输入方阵是否为对称矩阵（提示：比较A与A.T是否相等）。

习题 Q10 – Q18

Q10 用NumPy生成一个 4×4 矩阵A，编写代码验证A的主对角线元素在转置前后是否保持不变。

Q12 用随机数生成一个 3×4 矩阵A和一个 4×3 矩阵B，验证 $A + B.T$ 是否满足交换律，并解释为什么两者形状必须匹配才能相加。

Q14 用seaborn.heatmap()并排绘制矩阵A与其转置A.T，观察两幅热图之间颜色排列的镜像关系。

Q16 用随机数生成一个 8×8 矩阵A，编写代码验证 `numpy.diag(A)` 与 `numpy.diag(A.T)` 是否始终相等，并解释原因。

Q18 用随机数生成两个形状相同的矩阵A、B，验证 $(A - B).T$ 是否等于 $A.T - B.T$ 。

Q11 编写函数`row_to_col(v)`，输入一个一维行向量，返回其转置后的列向量（提示：可用`reshape`或`[:,None]`）。

Q13 编写代码验证：一个细高矩阵（行数>列数）转置后一定是扁平矩阵（行数<列数），并用随机矩阵测试至少3组不同形状。

Q15 编写函数`is_row_vector(x)`和`is_col_vector(x)`，判断输入的NumPy数组是否分别为行向量或列向量。

Q17 编写函数`transpose_by_loop(A)`，不调用A.T或`numpy.transpose()`，仅用双重for循环手动实现矩阵转置，并与A.T的结果对比验证。

习题 Q19 – Q26

Q19 用NumPy构造一个 5×5 的单位矩阵 (`numpy.eye(5)`)，验证其转置是否等于自身，并说明这属于哪一类特殊矩阵。

Q21 用随机数生成一个 3×3 矩阵A，验证矩阵乘法与转置的关系：
`A @ A.T` 是否一定是对称矩阵。

Q23 用Pandas读取鸢尾花数据集，转换为NumPy矩阵后对其做转置，并说明转置前后“样本”和“特征”这两个维度分别对应矩阵的哪个方向。

Q25 编写代码，验证转置操作是否改变NumPy数组占用的内存（提示：用`numpy.shares_memory()`检查A与A.T是否共享内存）。

Q20 编写代码，比较`numpy.transpose(A)`与`A.T`在结果和用法上是否完全一致。

Q22 编写函数`block_diag_check(blocks)`，输入若干方阵，构造一个分块对角矩阵，并验证其转置等于各分块转置后重新组成的分块对角矩阵。

Q24 用随机数生成一个 $2 \times 3 \times 4$ 的三维数组，尝试调用`numpy.transpose(arr, axes=(1,0,2))`，观察并解释多维数组转置与二维矩阵转置的区别。

Q26 综合运用本节知识，编写小程序：输入任意矩阵，依次输出其转置矩阵、判断是否为方阵、判断是否为对称矩阵，以及验证双重转置是否恢复原矩阵。

10 个可深入探讨的 Prompt (1/2)

- 1 请解释为什么本书用 A^T 而不是 A' 来表示转置，这与NumPy的.T记法有什么关系？
- 2 请用生活中的例子解释“转置不是镜像”这件事，比如把一张课程表的“星期”和“节次”对调。
- 3 请证明：对称矩阵一定是方阵；反过来，方阵是否一定对称？请举反例。
- 4 为什么 $(A+B)^T = A^T+B^T$ 成立，但 $(AB)^T$ 一般不等于 A^TB^T ？（提示：可以先思考后续矩阵乘法一节）
- 5 请解释主对角线元素为什么在转置前后位置不变，这对判断对称矩阵有什么帮助？

10 个可深入探讨的 Prompt (2/2)

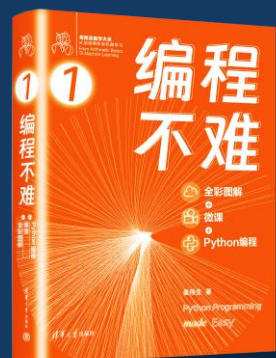
- 6 请设计一个Python小实验，验证细高矩阵与扁平矩阵在转置后的形状互换关系。
- 7 请解释图9、图10中几何视角下，矩阵的行向量与列向量转置前后分别落在几维空间中，为什么维度会互换？
- 8 请举一个数据科学场景，说明为什么经常需要对数据矩阵做转置（比如样本×特征与特征×样本之间的切换）。
- 9 请解释误区(b) “转置不是左右镜像”背后的原因：为什么第1列转置后一定是第1行，而不是移到最右侧？
- 10 如果一个矩阵满足 $A = -A^T$ （反对称矩阵），它的主对角线元素会有什么特点？请用Python验证你的猜想。

数学

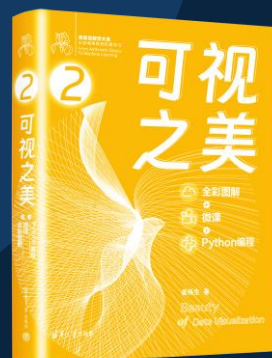
数学基础

线性代数

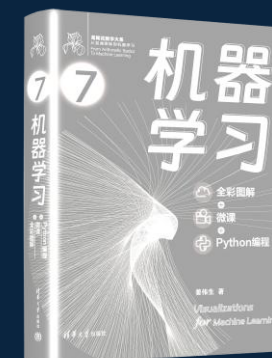
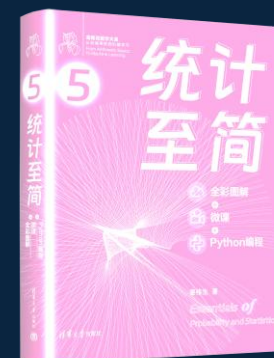
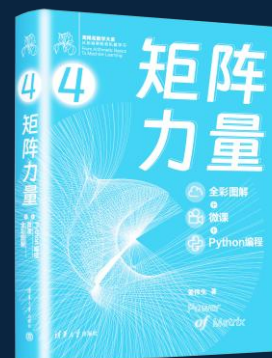
概率统计



Python编程



数据可视化



工具

数据分析
图和网络

回归、降维
分类、聚类

实践

